

Comprehensive Multi-threading & Process Management Notes

1. Thread vs Process

Process

- **Definition**: An independent execution unit with its own memory space
- **Characteristics**:
 - Has dedicated virtual memory space
 - Contains program code, data, heap, and stack
 - Independent execution context
 - Communication through IPC mechanisms

Examples:

- Running Chrome browser (separate process)
- VS Code editor (separate process)
- Calculator application (separate process)

Thread

- **Definition**: A lightweight execution unit within a process
- **Characteristics**:
 - Shares memory space with other threads in same process
 - Has own stack and register set
 - Shares code, data, and heap segments
 - Faster creation and context switching

Examples:

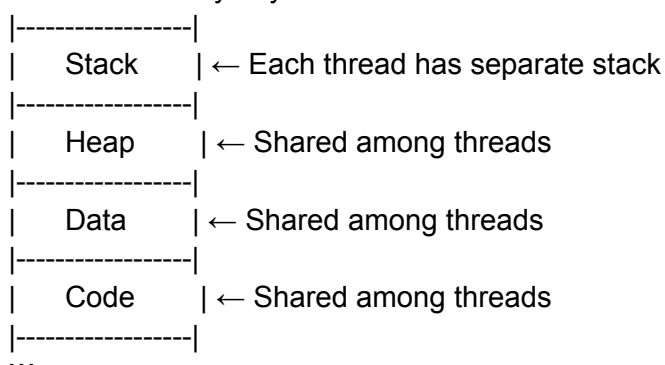
- Multiple tabs in Chrome browser (threads within Chrome process)
- Background spell-checker in text editor
- Download manager threads in browser

Step-by-Step Understanding:

1. **Memory Layout**:

...

Process Memory Layout:



2. **Creation Process**:

- **Process**: OS allocates new memory space → loads program → creates PCB

- **Thread**: OS creates TCB → allocates stack space → shares existing memory

🎯 Exam Angle Answers:

Q: Difference between process and thread?

Aspect	Process	Thread
Memory	Independent memory space	Shares memory with other threads
Communication	IPC (pipes, sockets, shared memory)	Direct memory access
Creation Cost	High (memory allocation)	Low (only stack allocation)
Context Switch	Expensive (memory context change)	Cheaper (register context only)
Isolation	High (crash doesn't affect others)	Low (one thread crash affects all)
Scheduling	By OS scheduler	By OS scheduler

Q: Why are threads called lightweight processes?

Threads are called "lightweight processes" because:

- Resource Sharing**: Unlike processes, threads share most resources (memory, files, I/O)
- Faster Creation**: Thread creation doesn't require new memory space allocation
- Efficient Communication**: No need for complex IPC mechanisms
- Lower Overhead**: Context switching between threads is faster
- Similar Functionality**: Both can execute independently and be scheduled by OS

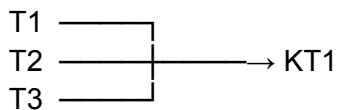
2. Thread Models

Step-by-Step Understanding:

Many-to-One Model

...

User Threads Kernel Thread



...

Process Flow:

- Multiple user threads created
- Thread library manages all threads
- OS sees only one kernel thread
- All user threads mapped to single kernel thread

Advantages: Fast thread switching, efficient

Disadvantages: No true parallelism, blocking system call blocks all threads

One-to-One Model

...

User Threads Kernel Threads

T1 —————→ KT1

T2 —————→ KT2

T3 —————→ KT3

...

****Process Flow**:**

1. Each user thread maps to kernel thread
2. OS manages each thread independently
3. True parallelism achieved
4. Used by Linux and Windows

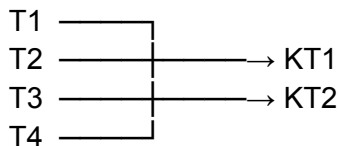
****Advantages**:** True parallelism, one blocked thread doesn't block others

****Disadvantages**:** Higher overhead, limited by kernel thread limit

Many-to-Many Model

...

User Threads Kernel Threads



...

****Process Flow**:**

1. Multiple user threads
2. Smaller number of kernel threads
3. Dynamic mapping based on need
4. Best of both worlds approach

🎯 Exam Angle Answer:

****Q: Which model does Linux/Windows follow?****

- ****Linux**:** One-to-One model using Native POSIX Thread Library (NPTL)
- ****Windows**:** One-to-One model
- ****Solaris**:** Many-to-Many model (Green threads)

3. Thread Libraries

Step-by-Step Classification:

User-Level Threads

...

Application

└─ Thread Library (POSIX pthread, Java threads)

```

|— User Threads (T1, T2, T3)
|— System Calls Interface
|   |— Kernel (sees single process)
...

```

****Working Process**:**

1. Application creates threads using library
2. Library maintains thread table
3. Library handles scheduling and switching
4. Kernel unaware of individual threads

Kernel-Level Threads

...

Application

```

|— System Calls (clone(), CreateThread())
|— Kernel Thread Manager
|— Kernel Threads (KT1, KT2, KT3)
...

```

****Working Process**:**

1. Application makes system calls
2. Kernel creates and manages threads
3. OS scheduler handles thread scheduling
4. Each thread visible to kernel

Hybrid Approach

...

Application

```

|— User Thread Library
|— User Threads —→ Kernel Threads
|— Kernel Management
...

```

🎯 Exam Angle Answer:

****Q: Explain user-level vs kernel-level threads.****

Feature	User-Level	Kernel-Level
Management	Thread library manages	OS kernel manages
Visibility	Invisible to OS	Visible to OS
Context Switch	Fast (no kernel involvement)	Slow (kernel involvement)
Blocking	One blocked thread blocks all	Individual thread blocking
Multiprocessing	Cannot utilize multiple CPUs	Can utilize multiple CPUs
Creation Cost	Low	High
Examples	Green threads, GNU Pth	Linux NPTL, Windows threads

4. Thread Scheduling

Step-by-Step Process:

Preemptive Scheduling

...

Timeline: |----T1----|--T2--|----T1----|--T3--|
 ^ ^ ^ ^
 Start Timer Resume Timer
 Int. Int.

...

Working Flow:

1. Thread starts execution
2. Timer interrupt occurs (time quantum expired)
3. OS scheduler saves current thread state
4. Selects next thread from ready queue
5. Loads new thread context and executes
6. Process repeats

Examples:

 Round Robin, Priority Scheduling

Cooperative Scheduling

...

Timeline: |-----T1-----yield--|--T2--yield--|
 ^ ^ ^
 Start T1 yields T2 yields

...

Working Flow:

1. Thread executes until it voluntarily yields
2. Thread calls yield() or similar function
3. Control returns to scheduler
4. Scheduler selects next thread
5. No forced preemption

Examples:

 Windows 3.1, Early Mac OS

🎯 Exam Angle Answers:

Q: Explain difference between preemptive and cooperative scheduling with examples.

Aspect	Preemptive	Cooperative
Control	OS forces thread switch	Thread voluntarily yields
Timer	Uses timer interrupts	No timer needed
Response Time	Better (guaranteed time slices)	Poor (depends on thread behavior)
Fairness	Fair sharing	Unfair (greedy threads can dominate)

| **Overhead** | Higher (timer interrupts) | Lower (no interrupts) |
| **Examples** | Linux, Windows NT+ | Windows 3.1, Older systems |

Q: Why does Linux use preemptive scheduling for threads?

Linux uses preemptive scheduling because:

1. **Fairness**: Ensures all threads get CPU time
2. **Responsiveness**: Prevents one thread from monopolizing CPU
3. **Real-time Support**: Guarantees bounded response times
4. **System Stability**: Prevents system hangs from infinite loops
5. **Multi-user Environment**: Fair resource sharing among users

5. Synchronization Problems

Step-by-Step Understanding:

Race Condition

...

```
Thread 1:      Thread 2:
counter = load_counter()
              counter = load_counter()
counter = counter + 1
              counter = counter + 1
store_counter(counter)
              store_counter(counter)
...
```

Problem Flow:

1. Both threads read same initial value
2. Both increment locally
3. Both store same result
4. One increment is lost!

Critical Section

...

```
do {
    [Entry Section]  ← Request entry

    /* Critical Section */ ← Only one thread allowed

    [Exit Section]   ← Release access

    /* Remainder Section */
} while (true);
...
```

****Requirements**:**

1. ****Mutual Exclusion****: Only one thread in critical section
2. ****Progress****: Decision to enter cannot be postponed indefinitely
3. ****Bounded Waiting****: Limit on times others enter before waiting thread

Mutex Implementation

```
```c
// Mutex operations
mutex_lock(&mutex);
 /* Critical Section */
mutex_unlock(&mutex);
```
```

****Working Process**:**

1. Thread calls `mutex\_lock()`
2. If mutex available → acquire lock, enter critical section
3. If mutex locked → thread blocks/waits
4. Thread in critical section calls `mutex\_unlock()`
5. Waiting threads compete for lock

Semaphore Implementation

```
```c
// Semaphore with count = n
wait(semaphore) { // P operation
 while (semaphore <= 0);
 semaphore--;
}

signal(semaphore) { // V operation
 semaphore++;
}
```
```

****Types**:**

- ****Binary Semaphore****: Count = 0 or 1 (like mutex)
- ****Counting Semaphore****: Count = n (resource pool)

Monitor

```
```java
public class Monitor {
 private int data;

 public synchronized void method1() {
 // Only one thread can execute
 // any synchronized method at a time
 }

 public synchronized void method2() {
```

```

 // Automatic mutual exclusion
 }
}
...

```

### ### 🎯 Exam Problems with Solutions:

#### ### Dining Philosophers Problem

##### \*\*Problem Setup\*\*:

- 5 philosophers around circular table
- 5 chopsticks (one between each pair)
- Need 2 chopsticks to eat
- Can lead to deadlock if all pick up left chopstick simultaneously

##### \*\*Solution using Semaphores\*\*:

```

```c
semaphore chopstick[5] = {1,1,1,1,1}; // Binary semaphores
semaphore mutex = 1; // For critical section

void philosopher(int i) {
    while(true) {
        think();

        wait(mutex); // Ensure atomic pickup
        wait(chopstick[i]);
        wait(chopstick[(i+1)%5]);
        signal(mutex);

        eat();

        signal(chopstick[i]);
        signal(chopstick[(i+1)%5]);
    }
}
...

```

Producer-Consumer Problem

Problem:

- Producer creates items, puts in buffer
- Consumer takes items from buffer
- Buffer has limited size
- Need synchronization

Solution:

```

```c
semaphore full = 0; // Count of full slots

```



```
semaphore empty = n; // Count of empty slots
semaphore mutex = 1; // Mutual exclusion
```

```
void producer() {
 while(true) {
 item = produce_item();

 wait(empty); // Wait for empty slot
 wait(mutex); // Enter critical section
 add_item(item); // Add to buffer
 signal(mutex); // Exit critical section
 signal(full); // Signal new full slot
 }
}
```

```
void consumer() {
 while(true) {
 wait(full); // Wait for full slot
 wait(mutex); // Enter critical section
 item = remove_item(); // Remove from buffer
 signal(mutex); // Exit critical section
 signal(empty); // Signal new empty slot

 consume_item(item);
 }
}
...
```

### ### Readers-Writers Problem

**\*\*Problem\*\*:**

- Multiple readers can read simultaneously
- Only one writer can write at a time
- No reading while writing

**\*\*Solution\*\*:**

```
```c
semaphore resource = 1; // Access to shared resource
semaphore readMutex = 1; // Mutual exclusion for readcount
int readcount = 0;    // Number of readers

void reader() {
    wait(readMutex);
    readcount++;
    if(readcount == 1)
        wait(resource); // First reader locks resource
    signal(readMutex);
```

```

/* Reading */

wait(readMutex);
readcount--;
if(readcount == 0)
    signal(resource); // Last reader unlocks resource
signal(readMutex);
}

void writer() {
    wait(resource); // Exclusive access

    /* Writing */

    signal(resource);
}
...

```

6. Deadlock

Step-by-Step Understanding:

Coffman's Conditions (Necessary for Deadlock):

1. **Mutual Exclusion**

...

Resource R1 can be used by only one process at a time

P1 holds R1 exclusively

...

2. **Hold and Wait**

...

P1 holds R1 and waits for R2

P1: Hold(R1) + Wait(R2)

...

3. **No Preemption**

...

R1 cannot be forcibly taken from P1

Only P1 can release R1 voluntarily

...

4. **Circular Wait**

...

P1 → waits for → P2

P2 → waits for → P3

P3 → waits for → P1
Forms a cycle!
...

Deadlock Example:

```
...  
Process P1:      Process P2:  
lock(mutex1);    lock(mutex2);  
lock(mutex2);    lock(mutex1); ← Deadlock!  
// use resources // use resources  
unlock(mutex2);  unlock(mutex1);  
unlock(mutex1);  unlock(mutex2);  
...
```

Deadlock Handling Strategies:

1. Deadlock Prevention

****Break at least one Coffman condition**:**

- ****Break Mutual Exclusion****: Make resources shareable (not always possible)
- ****Break Hold and Wait****:

```
``c  
// Acquire all resources at once  
lock(mutex1);  
lock(mutex2);  
// use both resources  
unlock(mutex2);  
unlock(mutex1);  
...
```

- ****Break No Preemption****: Allow resource preemption
- ****Break Circular Wait****: Order resources, acquire in order

```
``c  
// Always acquire lower-numbered resource first  
if (mutex1_id < mutex2_id) {  
    lock(mutex1); lock(mutex2);  
} else {  
    lock(mutex2); lock(mutex1);  
}  
...
```

2. Deadlock Avoidance - Banker's Algorithm

****Step-by-Step Process**:**

1. ****Data Structures****:

```
...  
Available[m]: Available instances of each resource  
Max[n][m]: Maximum demand of each process
```

Allocation[n][m]: Currently allocated resources
 Need[n][m]: Remaining resource need = Max - Allocation
 ...

2. **Safety Algorithm**:

...
 1. Work = Available
 2. Finish[i] = false for all i
 3. Find i such that: Finish[i] = false AND Need[i] ≤ Work
 4. Work = Work + Allocation[i]; Finish[i] = true
 5. Repeat steps 3-4 until all Finish[i] = true (Safe State)
 ...

3. **Resource Request Algorithm**:

...
 If Request[i] ≤ Need[i] AND Request[i] ≤ Available:
 Pretend to allocate resources
 Run safety algorithm
 If safe: make allocation
 Else: make process wait
 ...

Example:

...

Processes: P0, P1, P2

Resources: A=10, B=5, C=7

	Allocation			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	3	3	2
P1	2	0	0	3	2	2	1	2	2			
P2	3	0	2	9	0	2	6	0	0			

Safe Sequence: P1 → P0 → P2 (System is in safe state)

...

3. Deadlock Detection & Recovery

Detection Algorithm:

...

1. For each process, if Allocation ≠ 0, set Finish = false
 2. Find process Pi: Finish[i] = false AND Request[i] ≤ Available
 3. Available = Available + Allocation[i]; Finish[i] = true
 4. If all Finish[i] = true, no deadlock
 Else, deadlock exists among Finish[i] = false processes
 ...

Recovery Methods:

1. **Process Termination**: Kill deadlocked processes
2. **Resource Preemption**: Take resources from processes

🎯 Exam Angle Answers:

Q: State and explain necessary conditions for deadlock.

The four necessary conditions (Coffman's conditions) are:

1. **Mutual Exclusion**: Resources cannot be shared simultaneously
 - **Example**: Printer can be used by only one process
2. **Hold and Wait**: Process holds resources while waiting for others
 - **Example**: Process has file lock, waiting for printer
3. **No Preemption**: Resources cannot be forcibly removed
 - **Example**: Cannot take printer away from printing process
4. **Circular Wait**: Chain of processes waiting for each other
 - **Example**: $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ (circular dependency)

All four conditions must be present simultaneously for deadlock to occur.

Q: Which algorithm is used to avoid deadlock in resource allocation?

Banker's Algorithm is used for deadlock avoidance:

- **Purpose**: Determine if resource allocation leads to safe state
- **Principle**: Only allocate resources if system remains in safe state
- **Safe State**: System can complete all processes without deadlock
- **Working**: Simulates allocation, checks if safe sequence exists
- **Time Complexity**: $O(m \times n^2)$ where m =resources, n =processes

7. Context Switching

Step-by-Step Process:

Process Context Switch:

...

1. **Save Current Process State**:
 - CPU registers $\rightarrow$ PCB
 - Program counter $\rightarrow$ PCB
 - Stack pointer $\rightarrow$ PCB
 - Memory management info $\rightarrow$ PCB
2. **Update Process State**:

- Current process: Running → Ready/Waiting
- Next process: Ready → Running

3. Switch Memory Context:

- Save memory mappings
- Load new process memory mappings
- Flush TLB (Translation Lookaside Buffer)

4. Load Next Process State:

- PCB → CPU registers
- PCB → Program counter
- PCB → Stack pointer

5. Resume Execution:

- Jump to new process instruction

...

Thread Context Switch:

...

1. Save Current Thread State:

- CPU registers → TCB
- Program counter → TCB
- Stack pointer → TCB

2. Update Thread State:

- Current thread: Running → Ready/Waiting
- Next thread: Ready → Running

3. Load Next Thread State:

- TCB → CPU registers
- TCB → Program counter
- TCB → Stack pointer

4. Resume Execution:

- Jump to new thread instruction

Note: No memory context switch needed (shared memory)

...

Context Switch Cost Analysis:

...

Direct Costs:

- Save/restore registers: ~100 CPU cycles
- Kernel overhead: ~1000 CPU cycles
- TLB flush (processes): ~1000 CPU cycles

Indirect Costs:

- Cache misses: ~10,000+ CPU cycles

- Memory hierarchy effects
- Pipeline stalls
- ...

🎯 Exam Angle Answers:

****Q: Why is context switching expensive?****

Context switching is expensive due to:

- **Direct Overhead**:**
 - Saving current process/thread state
 - Loading new process/thread state
 - Kernel mode transitions (user→kernel→user)
- **Memory Management Overhead** (for processes):**
 - Memory mapping changes
 - TLB (Translation Lookaside Buffer) flush
 - Page table switching
- **Cache Effects**:**
 - Cache pollution (new process data replaces old)
 - Cache misses increase significantly
 - Cold cache effects for new process
- **Pipeline Effects**:**
 - CPU pipeline flush and restart
 - Branch prediction table reset

****Q: Difference between process context switch and thread context switch?****

Aspect	Process Context Switch	Thread Context Switch
-----	-----	-----
Memory Context	Must switch memory mappings	No memory switch (shared)
TLB Flush	Required (different address space)	Not required
Cache Impact	High (different memory layout)	Lower (shared memory)
Time Cost	1-10 microseconds	0.1-1 microseconds
Kernel Involvement	High overhead	Lower overhead
Saved State	Complete PCB	Only registers and stack

8. Thread Creation in Linux

Step-by-Step Process:

Linux clone() System Call:

```
```c
```

```
#include <sched.h>
```

```
pid_t clone(int (*fn)(void *), void *child_stack,
 int flags, void *arg, ...);
...
```

**\*\*Flags for Thread Creation\*\*:**

```
```c  
CLONE_VM    // Share virtual memory  
CLONE_FS    // Share file system info  
CLONE_FILES // Share file descriptor table  
CLONE_SIGHAND // Share signal handlers  
CLONE_THREAD // Create thread (not process)  
CLONE_SYSVSEM // Share SysV semaphores  
```
```

**\*\*Thread Creation Process\*\*:**

```
...

1. Application calls pthread_create()
2. pthread_create() calls clone() with appropriate flags:
 clone(thread_function, stack_ptr,
 CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND, arg)
3. Kernel creates new task_struct
4. New task shares memory, files, signal handlers with parent
5. New task gets separate stack and registers
6. Scheduler adds task to ready queue
...
```

**### pthread\_create() Implementation:**

```
```c  
#include <pthread.h>  
  
int pthread_create(pthread_t *thread,      // Thread ID  
                  const pthread_attr_t *attr, // Thread attributes  
                  void *(*start_routine)(void*), // Thread function  
                  void *arg);              // Function argument  
  
// Example usage:  
pthread_t thread_id;  
int result = pthread_create(&thread_id, NULL, worker_function, &data);  
...
```

Internal Implementation Flow:

```
...  
  
pthread_create() {  
    1. Allocate stack space for new thread  
    2. Set up thread attributes (stack size, priority, etc.)  
    3. Call clone() with thread-specific flags  
}
```



```

    4. Store thread ID in pthread_t structure
    5. Return thread ID to caller
}

```

```

Kernel clone() {
    1. Create new task_struct
    2. Copy/share resources based on flags
    3. Initialize registers and stack pointer
    4. Add to scheduler ready queue
    5. Return thread/task ID
}
...

```

🎯 Exam Angle Answer:

****Q: Which system call does Linux use internally for thread creation?****

****Answer: Linux uses `clone()` system call internally for thread creation.****

****Explanation**:**

- `pthread\_create()` is a POSIX library function (user-level)
- Internally, `pthread\_create()` calls `clone()` system call (kernel-level)
- `clone()` can create both processes and threads based on flags
- For threads: `clone()` called with flags like `CLONE\_VM`, `CLONE\_FILES`, etc.
- This allows fine-grained control over what resources are shared
- Linux treats threads as "tasks" that happen to share memory space

****Key Point**:** Linux doesn't distinguish between processes and threads at kernel level - both are "tasks" with different sharing characteristics.

9. Process States

Step-by-Step State Transitions:

Process Life Cycle:

...

NEW → READY → RUNNING → WAITING/BLOCKED → TERMINATED



...

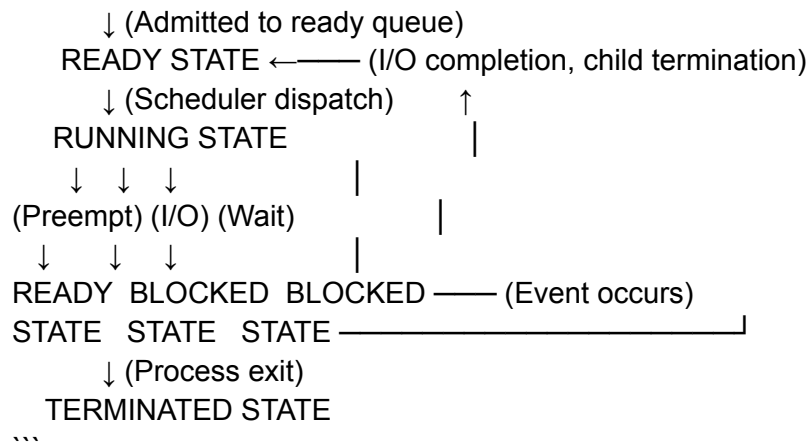
Detailed State Diagram:

...

[Process Created]



NEW STATE



State Descriptions:

1. ****NEW****:

- ...
- Process is being created
 - Memory allocated, PCB created
 - Not yet in ready queue
 - Initialization in progress
- ...

2. ****READY****:

- ...
- Process is ready to execute
 - Waiting for CPU allocation
 - In ready queue
 - All resources available except CPU
- ...

3. ****RUNNING****:

- ...
- Process is executing on CPU
 - Currently has CPU control
 - Can transition to ready, blocked, or terminated
- ...

4. ****WAITING/BLOCKED****:

- ...
- Process waiting for event/resource
 - Cannot proceed until event occurs
 - Examples: I/O completion, semaphore signal
- ...

5. ****TERMINATED****:

- ...
- Process has finished execution

- Resources being deallocated
- PCB may persist temporarily (zombie state)
- ...

Thread State Transitions:

...

Similar to processes but within same memory space:

THREAD_NEW → THREAD_READY → THREAD_RUNNING → THREAD_BLOCKED →
THREAD_TERMINATED

...

State Transition Triggers:

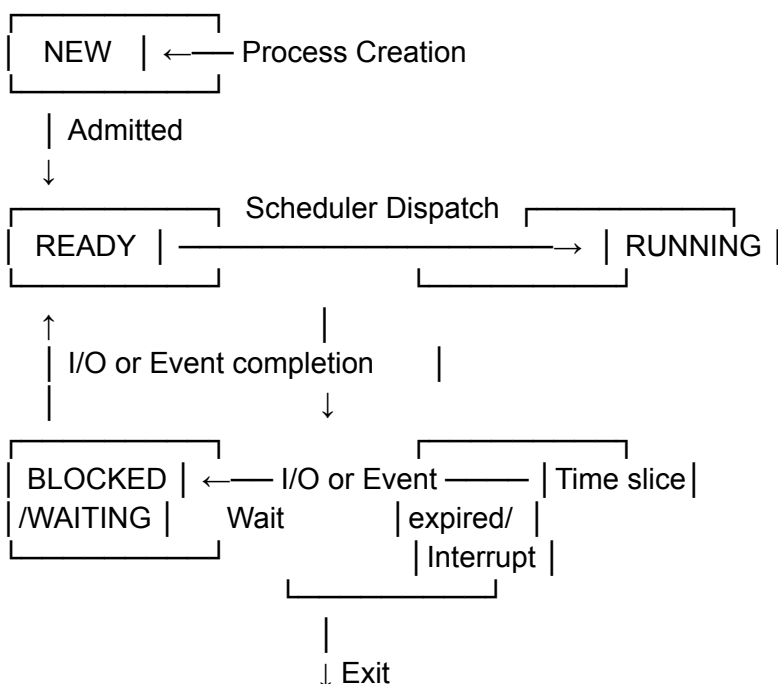
Transition	Trigger	Example
----- ----- -----		
New → Ready	Process admitted	OS loads program
Ready → Running	CPU scheduling	Round-robin time slice
Running → Ready	Preemption	Timer interrupt
Running → Blocked	Resource wait	File I/O request
Blocked → Ready	Resource available	I/O completion
Running → Terminated	Process exit	main() returns

🎯 Exam Angle Answers:

****Q: Draw and explain process state transition diagram.****

...

Process State Transition Diagram:



TERMINATED

...

****Explanation of Transitions**:**

1. ****New → Ready****: Process admitted to system
2. ****Ready → Running****: CPU scheduler selects process
3. ****Running → Ready****: Time quantum expired or preemption
4. ****Running → Blocked****: Process requests I/O or waits for event
5. ****Blocked → Ready****: I/O completed or event occurred
6. ****Running → Terminated****: Process completes execution

****Q: Difference between ready state and waiting state.****

Aspect	Ready State	Waiting/Blocked State
Definition	Ready to execute, waiting for CPU	Waiting for resource/event
CPU Need	Needs CPU only	Cannot use CPU even if available
Resource Status	All resources available	Missing required resource
Queue Location	Ready queue	Blocked/wait queue
Scheduling	Can be scheduled immediately	Cannot be scheduled
Duration	Short (until next time slice)	Variable (depends on event)
Examples	Loaded program ready to run	Waiting for disk I/O, user input

10. Multithreading Benefits & Issues

Step-by-Step Analysis:

Benefits of Multithreading:

1. Responsiveness

///

Single-threaded Application:

User Request → [Processing] → Response (Blocked UI)

Multi-threaded Application:

User Request → [UI Thread] → Immediate Response
→ [Worker Thread] → Background Processing

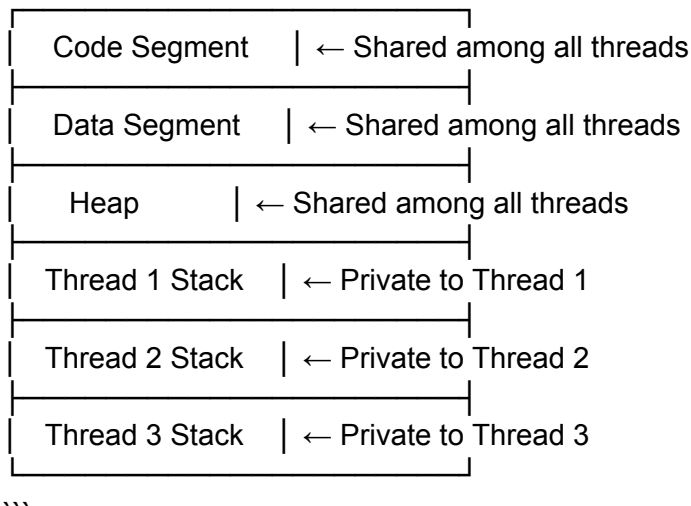
...

Example: Web browser downloading file while allowing navigation

2. Resource Sharing

...

Process Memory Layout with Threads:



****Benefits**:** Efficient memory usage, easy data sharing

3. Economy

...

Cost Comparison:

Process Creation:

- Memory allocation: High cost
- PCB creation: Medium cost
- Context switching: High cost

Thread Creation:

- Stack allocation: Low cost
- TCB creation: Low cost
- Context switching: Medium cost

Ratio: Thread creation ~10-100x faster than process creation

...

4. Scalability

...

Multi-core System Utilization:

Single-threaded: Uses 1 core only (25% utilization on 4-core)

Multi-threaded: Uses multiple cores (near 100% utilization)

Performance Formula:

$$\text{Speedup} = 1 / ((1-P) + P/N)$$

Where P = parallel portion, N = number of cores

...

❌ Issues with Multithreading:

1. Synchronization Overhead

...

Without Synchronization:

Thread execution: Direct memory access (fast)

With Synchronization:

Thread execution: Lock → Access → Unlock (slower)

Overhead includes:

- Lock acquisition/release time
- Blocking/unblocking threads
- Context switching due to blocking
- ...

2. Deadlocks

...

Classical Deadlock Scenario:

Thread A: Lock(R1) → Lock(R2)

Thread B: Lock(R2) → Lock(R1)

Result: Both threads blocked indefinitely

...

3. Context Switching Cost

...

Thread Context Switch: